

Project Proposal

Project Information

Title: A YOLO-based Multi-Object Detection Algorithm for E-bikes Carrying Passengers

Client: CJLU Traffic Management Department

Team Members: Jintao Lu, Jinhao Wu, Pinzhen You, Yu Zhu

Mentor: Mengke Xu

Date & Version: [2025/10/17 – v1.0]

Executive Summary

During peak traffic hours, illegal passenger carrying and helmetless riding on electric bicycles pose serious road safety risks. To address the inefficiency of traditional manual monitoring, this project aims to develop a real-time analytical system based on the YOLO object detection algorithm, capable of automatically and accurately identifying e-bike violations such as passenger carrying and helmet non-compliance.

A customized image dataset will first be constructed, containing four key categories: “e-bike,” “driver,” “passenger,” and “helmet,” with precise annotation for each instance. Then, an advanced YOLO model will be trained and optimized to perform rapid and accurate multi-object detection on every video frame. For passenger violations, the system determines infractions by analyzing the spatial relationship between “driver,” “passenger,” and “e-bike” bounding boxes—if two or more human targets are associated with a single e-bike, it is classified as a passenger violation. For helmet violations, the system searches the upper region of the “driver” or “passenger” bounding box, and if no “helmet” target is detected, the case is flagged as non-compliance.

To enhance temporal consistency and tracking reliability in video streams, the design incorporates multi-object tracking algorithms such as ByteTrack, assigning unique IDs to each detected target. Through inter-frame trajectory association, the system ensures the continuity and reliability of violation judgments. The final goal of this project is to build a high-accuracy, real-time violation recognition system capable of effectively handling complex traffic surveillance videos during peak hours, contributing to the construction of a safer and smarter campus.

1 Terms of Reference

Background

The campus population density is extremely high, with mixed flows of people and vehicles during peak commuting hours. Electric bicycles, which already have limited stability,

become even more dangerous when carrying passengers illegally. This leads to reduced maneuverability and braking reliability, directly threatening student safety. Another critical safety concern is helmet use: riders often neglect to wear helmets, increasing the risk of severe injury in accidents. Furthermore, detection systems must operate under varied and challenging conditions such as nighttime, low lighting, or rainy weather, which can significantly impact visibility and recognition accuracy.

Client

The CJLU Traffic Management Department requires an AI-based solution to automatically detect unsafe e-bike behaviors (e.g., passenger carrying, helmet violations) and ensure robust performance in real-world environments.

Project Purpose

The purpose of this project is to develop a reliable and automated monitoring tool for detecting e-bike passenger violations and helmet non-compliance, while ensuring detection robustness across diverse scenarios (daytime, nighttime, rainy). This system will support traffic safety management and contribute to the university's smart campus initiative.

2 Rationale

The project addresses urgent safety hazards of passenger carrying and helmet violations while laying the foundation for broader extensions. By integrating detection, tracking, alerting, and logging into one system, the project offers a complete safety management loop. This approach not only benefits campus management but also provides a scalable reference for smart city traffic safety.

3 Objectives and Scope

Objectives

1. Train a YOLO model to detect e-bike passenger violations.
2. Extend detection to helmet compliance.
3. Develop a GUI-based demo system supporting real-time video stream inference.
4. Produce a detailed technical report on dataset preparation, training, and evaluation.

Scope

- **Functional requirements:** passenger detection, helmet detection, violation judgment, real-time video stream analysis, automatic logging (time, location, screenshots).
- **Non-functional requirements:** real-time inference (>20 FPS), accuracy $\geq 90\%$, 24/7 stable operation, modular extensibility (e.g., wrong-way riding, restricted zone entry), user and data security.

- **Infrastructure:** Alibaba Cloud GPU servers, CJLU lab resources, Python, PyTorch, TensorRT, Flask/Django GUI.

4 System Architecture

The proposed system adopts a four-layer modular architecture, consisting of the Presentation Layer, Application Layer, AI Engine Layer, and Data Layer. This structure ensures scalability, stability, and efficient data flow across components.

1. Presentation Layer (Web User Interface)

The system provides an intuitive web-based interface that serves as the unified platform for monitoring, reviewing, and managing detected violations. Instead of connecting to live surveillance cameras, the system simulates real-time monitoring by uploading multiple video streams recorded from different campus scenarios.

Key features include:

- **Video Upload and Real-time Simulation:** Users can upload multiple video clips representing various traffic scenarios (e.g., daytime, nighttime, rainy conditions). The system processes each video stream sequentially to simulate real-time detection and tracking.
- **Detection Visualization:** The interface displays each processed frame with bounding boxes and violation labels (e.g., helmetless rider, passenger carrying).
- **Violation Log View:** Automatically records violation details such as timestamps, frame IDs, and captured images for review and confirmation.
- **Analytics Dashboard:** Aggregates all detection results into visual statistics — including violation frequency, location-based trends, and compliance rates — presented through interactive charts and tables.
- **User Management:** Allows administrators to manage user accounts, system settings, and dataset uploads within the same portal.

This web-only design simplifies deployment and ensures accessibility from any browser, enabling both administrators and security officers to interact with the system efficiently without additional client installation.

2. Application Layer (Business Logic)

This layer handles system logic and communication between the interface and AI modules. It provides:

- Flask/Django API services for video stream input, data requests, and AI model invocation.
- User management and authentication, including account roles (admin, staff).
- Violation logging system, which records violation types, timestamps, locations, and associated images.

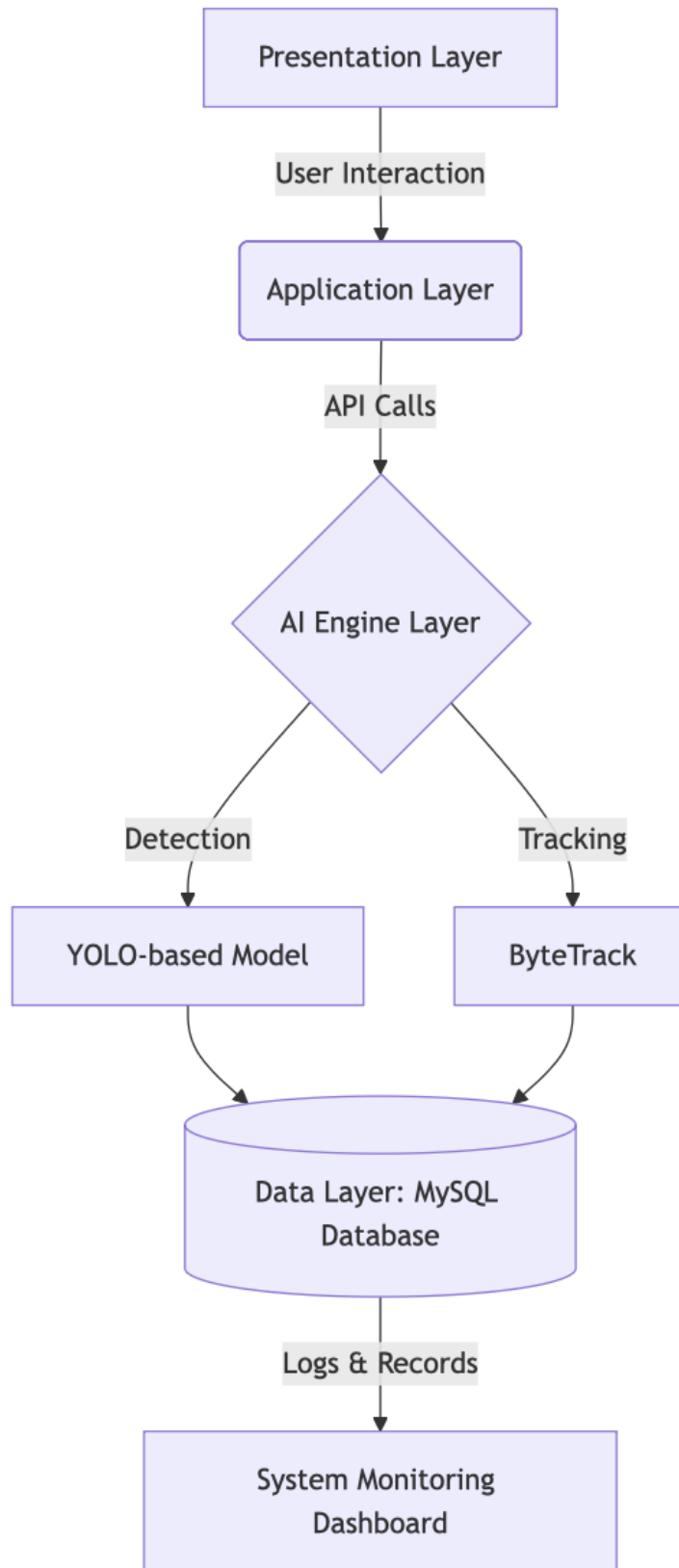


Figure 1: System Architecture Design

- Scheduling and alert modules, ensuring real-time message push to the UI or mobile devices.

3. AI Engine Layer (Detection and Tracking)

The AI module is the core of the system. It integrates:

- A YOLO-based detection model (YOLOv11, with BiFPN and GLSA attention enhancements) for accurate object detection of e-bike, driver, passenger, and helmet.
- Rule-based violation reasoning:
 - Passenger violations are determined by associating multiple person detections with the same e-bike region.
 - Helmet violations are flagged when no helmet is detected in the upper part of a rider's bounding box.
- ByteTrack multi-object tracking algorithm, which assigns unique IDs and maintains object trajectories across frames, ensuring temporal consistency in real-time video streams.
- Support for GPU inference and TensorRT optimization, ensuring over 25 FPS real-time performance.

4. Data Layer (Storage and Logging)

- A MySQL database stores structured data such as user information, violation logs, and system configuration.
- Local cache and log management handle image and video clip storage for evidence retrieval.
- Data synchronization allows exporting summarized reports or connecting to campus safety systems.
- All data access is secured through encrypted communication and role-based authorization.

This architecture enables continuous, real-time operation and facilitates future expansion (e.g., adding new violation types such as wrong-way riding or restricted-area entry).

5 User Analysis and Use Cases

5.1 User Analysis

The proposed system will serve multiple user roles within a smart campus safety management environment. Each role has different levels of access, functionality, and interaction with the system.

Table 1: User Roles and Access Levels

User Type	Description	Main Responsibilities	Access Level
Administrator	Oversees the entire system operation and manages user permissions.	Configure detection thresholds, manage user accounts, review violation statistics, export reports.	Full access to backend and all management modules.
Security Officer (Operator)	Responsible for monitoring camera streams and handling real-time alerts.	View live detection results, verify alerts, capture screenshots, tag false positives, manage on-site responses.	Medium access: monitoring and logging modules.
Data Analyst / Supervisor	Evaluates long-term trends and system performance.	Access violation database, generate statistical reports, analyze behavioral patterns for policy decisions.	Read-only access to data visualization and reports.
General User / Student (future extension)	Campus users affected by the safety policy.	Receive educational warnings or safety notifications through mobile or web app.	Limited access (notification-only).

This classification allows role-based access control (RBAC) and supports secure multi-user operation. Administrators and security officers form the primary operational group in the initial deployment phase, while analytical and public-facing functions can be extended later.

5.2 Use Cases

Table 2: System Use Cases

Use Case	Primary Actor	Description	Expected Outcome
UC1: Real-time Violation Monitoring	Security Officer	The officer monitors live video feeds. When the system detects a violation (e.g., a helmetless rider or passenger), it triggers an alert and marks the frame.	An alert message is displayed, and the violation is logged into a database.
UC2: Violation Verification and Annotation	Administrator	An administrator reviews logged violations and can correct misclassifications (e.g., marking a false positive).	Improved dataset quality, making model retraining possible.
UC3: Statistical Report Generation	Data Analyst	An analyst retrieves monthly violation data and analyzes patterns based on time, location, and frequency.	Statistical charts are generated for management review.
UC4: System Configuration and Threshold Adjustment	Administrator	The administrator adjusts detection confidence thresholds or adds new camera sources.	Updated system parameters are stored, and detection accuracy is optimized.
UC5: Notification and Response (Extension)	Security Officer / General User	The system sends instant notifications (e.g., via a mobile app) when violations occur.	A prompt response or educational intervention is triggered.

Each use case corresponds to specific modules within the architecture (e.g., UC1-UC2 in Application & AI Engine layers; UC3-UC4 in Data and Admin layers). This mapping ensures traceability from system requirements to implementation.

6 Skills Analysis

This project requires a combination of AI model development, software engineering, and system integration skills. The table below summarizes the existing team capabilities and technical gaps:

Table 3: Skills Analysis Summary

Skill Area	Current Strengths	Application in Project	Potential Gaps / Learning Focus
Deep Learning & Computer Vision	Familiarity with CNNs and object detection frameworks (YOLOv5-v11)	Model training, data preprocessing, and evaluation	Fine-tuning YOLOv11 hyper-parameters, model optimization using TensorRT
Multi-Object Tracking	Basic understanding of tracking-by-detection pipelines	Implementing ByteTrack to maintain target identity across frames	Advanced data association logic and trajectory management
Backend Development (Flask/Django)	Experience in Python web frameworks	Building RESTful APIs for AI inference and logging systems	API performance optimization and asynchronous data streaming
Frontend & GUI Development (PyQt / Web)	Moderate experience in GUI and HTML visualization	Building monitoring dashboards and user interaction modules	Enhancing UI responsiveness and integrating live video feeds
Database & Data Management	Familiar with MySQL queries and ER design	Storing and retrieving violation data, user accounts, and configuration	Designing efficient indexing for high-frequency data writes
Real-time System Optimization	Basic knowledge of GPU acceleration	Ensuring real-time performance for video stream inference	Mastery of CUDA/TensorRT deployment for low-latency inference
Project Coordination & Agile Workflow	Weekly sprint planning and team collaboration experience	Iterative system development, testing, and review	Improving documentation and integration testing processes

In summary, the team already possesses the necessary foundation in deep learning, Python programming, and system design, but will focus on improving real-time data processing, GPU optimization, and robust API integration during the implementation phase.

7 Team Roles

- **Algorithm Engineer:** Model selection, training, hyperparameter tuning.
- **Data Analyst:** Dataset search, annotation quality control.
- **Programmer:** Code implementation, inference pipeline.
- **Software Developer:** Demo system, GUI integration, deployment.

8 Team Schedule (Part 1)

Weekly Commitment

Each member will dedicate approximately **12–15 hours per week**. These hours cover model development, dataset annotation, web application coding, documentation, and testing.

Meeting Schedule

- **Mentor meeting:** Once per week (to report progress, discuss technical challenges, and review deliverables).
- **Client meeting:** Once every two weeks (to validate requirements and obtain feedback).
- **Internal team sync:** Twice per week (short 30-minute stand-ups for coordination).

Planned Milestones

Table 4: Project Timeline

Week	Milestone	Expected Deliverables / Focus
Week 1-4 (October)	Requirement Analysis & Web App Initialization	Define project scope and architecture; set up web application (Flask + frontend); implement user login and video upload modules; prepare initial annotation tool interface.
Week 5-6 (November)	Initial Dataset Creation & Labeling (≈ 100 images)	Begin manual annotation for 4 target classes (e-bike, driver, passenger, helmet). Train a baseline YOLOv8/YOLOv11 model using the first 100 annotated images to verify workflow and adjust parameters.
Week 7-8 (Late November)	Web-Model Integration & Early Testing	Integrate trained model into the web platform for image-based testing; visualize detection results and collect feedback; continue dataset expansion and refinement.
Week 9-10 (December)	Mid-Project Review	Deliver a working image-detection prototype and violation logging feature. Present model performance metrics (Precision, Recall, F1-score) and demonstrate system architecture.
Week 11-13 (January-February)	Iterative Labeling Expansion (≈ 500 images total)	Continue dataset annotation to enrich diverse conditions (lighting, weather, occlusion). Retrain YOLO model with extended dataset; introduce data augmentation (low-light, rain synthesis).
Week 14-16 (March)	Video Stream Processing & Tracking Integration	Extend detection to frame-by-frame video inference; implement ByteTrack for consistent ID tracking; optimize inference speed using TensorRT or pruning.
Week 17-18 (April)	Full Web Application Integration & Testing	Integrate detection, tracking, violation logging, and dashboard visualization. Conduct internal functional tests and usability review.
Week 19-20 (May)	Final Evaluation & Submission	Conduct multi-scenario testing (daytime, nighttime, rainy); finalize dataset (500+ annotated images); compile technical report, submit final system, and deliver presentation.

Summary

Dataset annotation will be an iterative and continuous process. An initial batch of approximately 100 manually labeled images will support early model verification, with subsequent annotation rounds expanding the dataset to over 500 images to improve model robustness and generalization. This data refinement loop, comprising annotation, model retraining, and validation, ensures that detection performance continuously improves in accuracy and stability. In parallel, the web application will evolve from image-based testing to full video-stream simulation, allowing real-time-like monitoring and visualization of detection results.

9 Project Management Methodology

Methodology

The project will follow an Agile development methodology with 2-week sprints, enabling rapid iteration, continuous feedback, and adaptive planning. This approach suits the project's nature, where data labeling, model training, and system integration progress in parallel and require frequent refinement.

Key characteristics of the adopted Agile workflow include:

- **Iterative Development:** The project is divided into multiple short sprints (2 weeks each), during which specific goals—such as dataset expansion, YOLO model optimization, or web interface refinement—are targeted and reviewed.
- **Incremental Dataset Enhancement:** Data annotation will follow an active learning loop, where new samples are labeled after each model evaluation to progressively improve accuracy and robustness.
- **Parallel Module Development:** The AI module (YOLO + ByteTrack) and the Web application (Flask front-end, data visualization) are developed concurrently, with integration checkpoints at the end of each major sprint.
- **Sprint Review & Retrospective:** Each sprint concludes with a demonstration of progress (e.g., new model performance or UI feature) and a retrospective discussion to plan for the next sprint.
- **Version Control & Collaboration:** All code, datasets, and reports are managed via GitHub, ensuring traceability, reproducibility, and efficient collaboration between team members.

This Agile process ensures that the system evolves incrementally yet consistently, allowing early testing, stakeholder feedback, and risk mitigation throughout development.

Sprint Planning

Each sprint will begin with a planning session to assign tasks, and end with a sprint review + retrospective to evaluate deliverables and identify improvements.

Work Breakdown Structure (WBS)

1. Dataset Collection & Annotation

- Identify data sources (campus cameras, online datasets).
- Annotate e-bike, passenger, and helmet attributes.

2. Model Training & Evaluation

- Select YOLO variant (YOLOv8 baseline).
- Train baseline model, tune hyperparameters.
- Evaluate using Precision, Recall, F1-score.

3. System Development & GUI Integration

- Build inference pipeline.
- Develop GUI (Flask web app).
- Integrate real-time video stream processing.

4. Deployment & Testing

- Test model in varied scenarios (day/night, rainy).
- Optimize inference speed (>20 FPS).
- Finalize demo system and prepare deployment package.

Justification

Agile allows flexible handling of dataset and model updates, ensures frequent validation with the client, and provides incremental deliverables that reduce project risk.

10 Risk & Issues Management

The project faces several technical and practical risks. To manage them, the team will adopt preventive and mitigation strategies:

1. **Dataset insufficiency (especially helmet/no-helmet and nighttime/rainy cases)** → Supplement with public datasets (VisDrone, Helmet Detection Dataset) and conduct manual annotation for campus-specific scenes.
2. **Low-light and rainy conditions reduce accuracy** → Apply advanced augmentation (low-light enhancement, synthetic rain/snow effects) and use YOLO variants optimized for small/occluded objects.
3. **Helmet misclassification (hats or hairstyles mistaken for helmets)** → Introduce multi-class labeling (helmet, head, hat, others) and incorporate context-aware post-processing.
4. **Rider identification ambiguity (distinguishing between rider and nearby pedestrian)** → Combine human bounding boxes with e-bike bounding boxes; use spatio-temporal association to confirm rider identity.

5. **Multiple riders and overlapping vehicles** → Use multi-object tracking algorithms (ByteTrack, DeepSORT) and augment dataset with overlapping cases.
6. **Real-time performance risk (FPS < 20)** → Optimize with TensorRT, pruning, quantization, and model distillation.
7. **System stability risk (continuous 24/7 operation)** → Add automatic recovery and watchdog scripts to handle process crashes.
8. **Data privacy and security risk** → Ensure local storage, apply encryption (SHA-256 for user accounts, anonymization of faces), and restrict access by user roles.

11 Project Plan

Table 5: Project Plan Overview

Phase	Timeframe	Key Objectives	Expected Deliverables
Phase 1: Project Setup & System Design	Weeks 1-4 (October)	Conduct requirement analysis and literature review (YOLOv8/v11, ByteTrack); finalize dataset categories and video simulation approach; deploy Alibaba Cloud GPU environment; initialize web application framework.	Project architecture document, initial Flask web interface prototype.
Phase 2: Dataset Preparation & Baseline Model	Weeks 5-8 (November)	Collect and annotate the first batch of images (~100); establish a baseline YOLOv8/YOLOv11 model; verify end-to-end image detection workflow.	Initial labeled dataset; baseline model results integrated into the web interface.
Phase 3: Iterative Model Optimization & Integration	Weeks 9-13 (December-February)	Expand dataset to ~500 annotated images; retrain YOLO with BiFPN-GLSA and ASF-Dysample modules; integrate ByteTrack tracking; start real-time simulation via uploaded video streams.	Improved detection accuracy (>90% precision); early-stage video stream prototype.
Phase 4: System Development & Dashboard Implementation	Weeks 14-18 (March-April)	Complete web-based monitoring dashboard; implement violation logging, analytics visualization, and multi-scenario testing (day/night/rainy).	Fully integrated web application with detection and analytics modules.
Phase 5: Final Testing & Deliverables	Weeks 19-20 (May)	Conduct final validation, stability testing, and user evaluation; prepare final documentation and presentation.	Final demo system, technical report, and trained model package.

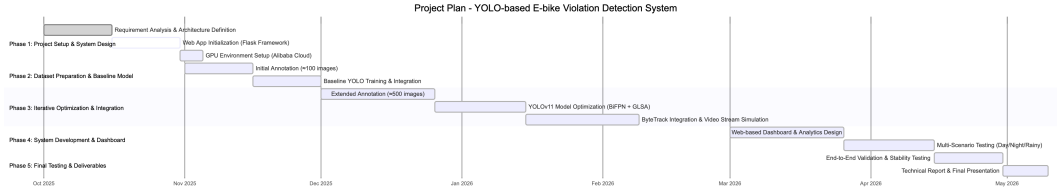


Figure 2: Project Timeline Visualization

Summary

The five-phase plan emphasizes progressive dataset enrichment, iterative model refinement, and early web integration. By aligning model development with interface design and validation milestones, the project ensures the system evolves coherently and remains testable throughout the implementation cycle.

12 Estimated Costs

GPU Resources

- **Primary:** Alibaba Cloud GPU instances for model training (pay-as-you-go). Estimated at ¥8–12/hour, with approximately 200–250 training hours required, leading to a projected cost of ¥2000–3000.
- **Backup:** Free platforms such as Google Colab for small-scale experiments and testing.

Labour Costs

- Mentor time @ \$142+GST per hour (university standard cost).
- Student hours: 3–4 students $\times$ 350 hours each = 1050–1400 hours (not monetized but documented).

Data Annotation Costs

- If external annotation support is needed, estimated at ¥0.5–1 per image, depending on dataset size. For 3000 images, this could total ¥1500–3000.

Additional Expenses

- Storage and network usage on Alibaba Cloud: approximately ¥100–200.
- Contingency (10% of total) to cover unexpected training reruns or additional GPU usage.

Total Estimated Project Cost

¥4000–6000 (excluding mentor hours).

13 Future Work / Extension

In the future, this project can be extended in several directions:

1. **Helmet Detection Enhancement:** Improve the accuracy of helmet detection by distinguishing between helmets, hats, and other head coverings, and extend the dataset with more diverse samples.
2. **Multi-Scenario Adaptation:** Conduct further experiments to improve robustness in adverse conditions, such as nighttime, rainy weather, or crowded traffic flows. This may include low-light image enhancement, weather simulation data augmentation, and multi-sensor fusion.
3. **Edge Deployment:** Optimize the model for deployment on embedded devices (e.g., NVIDIA Jetson, edge AI cameras) to enable real-time monitoring in campus environments without reliance on cloud GPUs.
4. **Integration with Campus Systems:** Explore integration with smart campus infrastructure, linking detection results to traffic management dashboards, automatic alerts, or safety reports.

These extensions will not only enhance the technical capabilities of the system but also increase its applicability and impact in real-world traffic safety management.